OrbitNav

A Self-Contained Attitude Determination and Control System (ADCS) Module for 3U–6U CubeSats

Preliminary Design Review (PDR)

Prepared by Orbit Corp

Presented by Camilo Garrido Fernandez

December 2025

Contents

1 Executive Summary **2**

2 System Architecture & Interfaces **3**
2.1 Sensor Integration 3
2.2 Actuator Integration 3

3 Estimation and Control Algorithms **5**
3.1 Estimation Algorithm — Kalman Filter 5
3.2 Control Algorithm — LQR Controller 5
3.2.1 Dynamics Overview 5
3.2.2 LQR Overview 6
3.2.3 LQR Controller Advantages 7

4 OrbitNav Implementation Diagram **8**

5 Performance and Feasibility **9**
5.1 Strategy for High Pointing Accuracy 9
5.2 Simulation 9
5.2.1 Simulation Overview 9
5.2.2 Simulation Result Examples 9

1 Executive Summary

The OrbitNav ADCS module from Orbit Corp is designed to provide robust, high-accuracy pointing and stabilization for 3U to 6U CubeSat platforms. But what truly sets OrbitNav apart is that it is envisioned as a plug-and-play solution, aimed to integrate seamlessly with a variety of hardware inputs while minimizing the number of parameters end-users need to specify for their system to a select few that are relatively easy to determine. The module itself would only consist of the flight processing chip running the OrbitNav software package, with a protoboard for users to connect their sensors and actuator controllers.

The attitude control software is generalized to work with any number and combination of reaction wheels, thrusters, and magnetorquers for actuating the CubeSat. It also provides users the ability to simulate their flight navigation with different hardware configurations to inform their hardware design choices. This way, OrbitNav can adapt to users' needs, rather than constraining them to its needs.

This Preliminary Design Review (PDR) details the fundamental system architecture, module integration process, algorithmic approach, and preliminary performance expectations of the OrbitNav system.

2 System Architecture & Interfaces

The OrbitNav flight computer is envisioned to be mounted on a protoboard with power and ground pins to connect to the CubeSat's onboard power supply, a set of input pins to receive sensor measurement data, and a set of output pins to command the actuator controllers. Users would be able to specify the number of input and output pins they need when placing their order for an OrbitNav module. The following sections detail the process for integrating sensors and actuators with the OrbitNav system.

2.1 Sensor Integration

The OrbitNav module can be integrated with any set of sensors that provide the following measurements:

- **Attitude** [rad] (Euler Angles)
- **3-axis Angular Velocities** [rad/s]
- **3-axis Local Magnetic Field** [T] (necessary if magnetorquers are used for actuation)

These can be achieved with a high-fidelity 3-axis gyro package (numerically integrating the angular velocities over time to track attitude) and magnetorquers — or paired with other sensors such as a Star Tracker or Earth and Sun sensor to measure attitude directly, or an inertial measurement unit, IMU, that packages gyros, accelerometers, and magnetometers.

Technically, a GPS receiver is not necessary for OrbitNav's attitude control program — though still advisable to track the CubeSat's location.

One important caveat is that the OrbitNav attitude control assumes zero-mean sensor noise. However, sensor measurements tend to gradually drift (i.e., accumulate errors over time). Thus, for best results, users should implement an algorithm to correct for drift first, and then pass those "corrected" measurements to the OrbitNav module. Hence, an intermediary computer to perform these corrections is recommended.

2.2 Actuator Integration

The OrbitNav module can be integrated with any number and configuration of the following actuators: **reaction wheels**, **thrusters**, **magnetorquers**. Users simply have to specify the parameters of these actuators — positions, orientations, actuation limits (i.e., max motor torque for reaction wheels, max thrust for the thrusters, and max current for the magnetorquers), etc — needed to calculate the CubeSat's dynamics. The OrbitNav module would come with a system configuration interface to facilitate this process.

During its state propagation loop, the OrbitNav module computes the desired actuator inputs: **Reaction wheel torques** [Nm], **Thruster desired output thrusts** [N], and **Magnetorquer torques** [Nm]. To command these actuators, users would write custom functions to specify which pins correspond to which actuators, and translate the desired inputs into real actuations. For instance, the reaction wheels are driven by motors with motor controllers that can convert the desired torque (T) into the current (I) commanded to the motor using the motor's

K_t ratio ($I = \frac{T_{des}}{K_t}$). Precise thrusts can be achieved using pulse-width modulations (PWM) commanded by the thruster controller. The OrbitNav module actually already calculates the desired magnetorquer current based on the desired torque and measured local magnetic field. This desired current would be passed directly to the magnetorquer coils. Each of these functions would be called during each step of the state propagation loop.

Importantly, the OrbitNav module already incorporates user-input actuator limits into its desired inputs — either by clamping individual actuations down to the user-input actuation limits, or by scaling down the collective actuations to preserve the resultant rotation direction while staying below the limits. Users can select which desaturation logic to implement when initiating the OrbitNav module. Thus, they would not need to worry about actuator limits when writing their simple actuator control functions.

Moreover, the OrbitNav system assumes that any thrusters used are bidirectional by default. However, single-directional thrusters can also be used. They would just require that users manually adjust the desaturation logic to clamp any negative desired-thrust values (calculated by the controller) to zero.

3 Estimation and Control Algorithms

3.1 Estimation Algorithm — Kalman Filter

The OrbitNav module implements a Kalman Filter to provide an optimal estimate of the spacecraft state by fusing the physical system dynamics model with noisy sensor measurements. This approach was selected based on several key design criteria:

- **Optimality:** Kalman filtering is mathematically proven to be the optimal (minimizing mean squared error) linear estimator for linear systems.
- **Computational Efficiency:** Because the system dynamics are modeled as Linear-Time Invariant (LTI), the Kalman Gain (L) can be pre-computed. This reduces the real-time computations to simple matrix additions and multiplications, ensuring that the flight computer can handle them.
- **Flexibility:** The Kalman filter can be generated for any hardware suite, provided the user can characterize the noise profile of their chosen components. By simply measuring and providing the standard deviations of sensor output and process disturbances, OrbitNav automatically computes the required noise covariance matrices (Q and R).
- **Robustness:** By weighting the "trust" between the physical dynamics model of the system and the sensor data, the filter prevents the controller from overreacting to transient sensor jitter.

3.2 Control Algorithm — LQR Controller

The control system employs a Linear Quadratic Regulator (LQR) controller, which allows users to define the costs of their inputs and the weights of their targets, and then automatically optimizes the control input (i.e., the desired actuation) to achieve the minimum cost solution.

3.2.1 Dynamics Overview

OrbitNav models the CubeSat dynamics using a linearized state-space model:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{w}$$
$$\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{v}$$

A : State Matrix (defines internal dynamics). Constructed as:

$$A = \begin{bmatrix} \mathbf{0}_{3\times3} & \mathbf{I}_{3\times3} \\ \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} \end{bmatrix}$$

B : Input Matrix (defines how inputs affect states). Constructed as:

$$B = \begin{bmatrix} \mathbf{0}_{3\times m} & & \\ -J^{-1}\Lambda_{rw} & J^{-1}\Lambda_{thr} & J^{-1} \end{bmatrix}$$

$\mathbf{m}$: The total number of control inputs.

$\mathbf{J^{-1}}$: The inverse of the CubeSat's total moment of inertia.

$\mathbf{\Lambda_{rw}}$: Maps scalar reaction wheel torques to vectorial torques based on wheel orientations and positions relative to the center of gravity.

$\mathbf{\Lambda_{thr}}$: Maps scalar input thrusts to resultant torques based on thruster moment arms relative to the center of gravity.

$\mathbf{C}$: Output Matrix (maps states to outputs). Constructed as:

$$C = \left[\mathbf{I}_{6\times 6}\right]$$

$\mathbf{x}$: The state vector representing attitude [rad] and angular rates [rad/s].

$$x = [\phi, \theta, \psi, p, q, r]^T$$

$\mathbf{u}$: The control input vector representing the torques [Nm] and forces [N] commanded of the actuators.

$$u = [T_{\text{reaction wheels}}, F_{\text{thrusters}}, T_{\text{magnetorquers}}]^T$$

$\mathbf{w}$: Process noise, representing disturbances in the satellite environment.

$\mathbf{v}$: Measurement noise, representing inherent sensor inaccuracies.

3.2.2 LQR Overview

The LQR controller works by minimizing a quadratic cost function J that balances pointing accuracy against control effort:

$$\mathbf{J} = \int_{\mathbf{0}}^{\infty} (\mathbf{x^T Q x} + \mathbf{u^T R u} + \mathbf{2 x^T N u})\mathbf{dt}$$

$\mathbf{Q}$: Penalty matrix for state errors (how much users care about accuracy).

$\mathbf{R}$: Penalty matrix for actuation (how expensive the inputs are).

$\mathbf{N}$: Coupling between state error and input cost. The OrbitNav module always assumes this to be $\mathbf{0}$ for simplicity, which ensures that the resulting feedback gain is derived purely from the user-specified relative importance of state convergence versus actuator conservation.

If users primarily care about attitude control, they can greatly decrease the weights of the angular velocities (or even make them zero). The costs of actuators can also be tuned. For instance, if users would rather expend current than thruster propellant, they could increase the thruster input costs relative to those of the magnetorquers and reaction wheels.

The resulting control command is calculated at each timestep via the state-feedback law:

$$\mathbf{u} = -\mathbf{K}(\hat{\mathbf{x}} - \mathbf{x_{target}})$$

$\mathbf{K}$: The LQR gain such that u minimizes J.

$\hat{\mathbf{x}}$: The optimal state estimate generated by the Kalman filter.

$\mathbf{x_{target}}$: The user-input desired state vector (i.e., target attitude and angular rates).

3.2.3 LQR Controller Advantages

To summarize, the LQR approach offers several critical advantages for 3U–6U CubeSat operations over a typical baseline (PID) controller:

- **Multiple Input Multiple Output (MIMO):** Unlike PID, which treats each axis (ϕ, θ, ψ) as an independent loop, LQR is a Multiple-Input Multiple-Output (MIMO) controller, meaning it accounts for the physical cross-coupling between axes. This facilitates coordinated maneuvers where an actuation in one axis automatically compensates for induced disturbances in another.
- **Optimal Efficiency:** LQR mathematically guarantees the most efficient path to the target state based on the user's defined state weights, $\mathbf{Q}$, and input costs, $\mathbf{R}$. This is vital for CubeSats because they always have to contend with limited power budgets and/or finite propellant.
- **Automated Tuning:** PID requires manual, trial-and-error tuning of separate gains with little (or at least convoluted) physical meaning. LQR replaces this with high-level weighting of physical priorities, which the module then uses to solve for the mathematically optimal gain matrix, $\mathbf{K}$.

4 OrbitNav Implementation Diagram

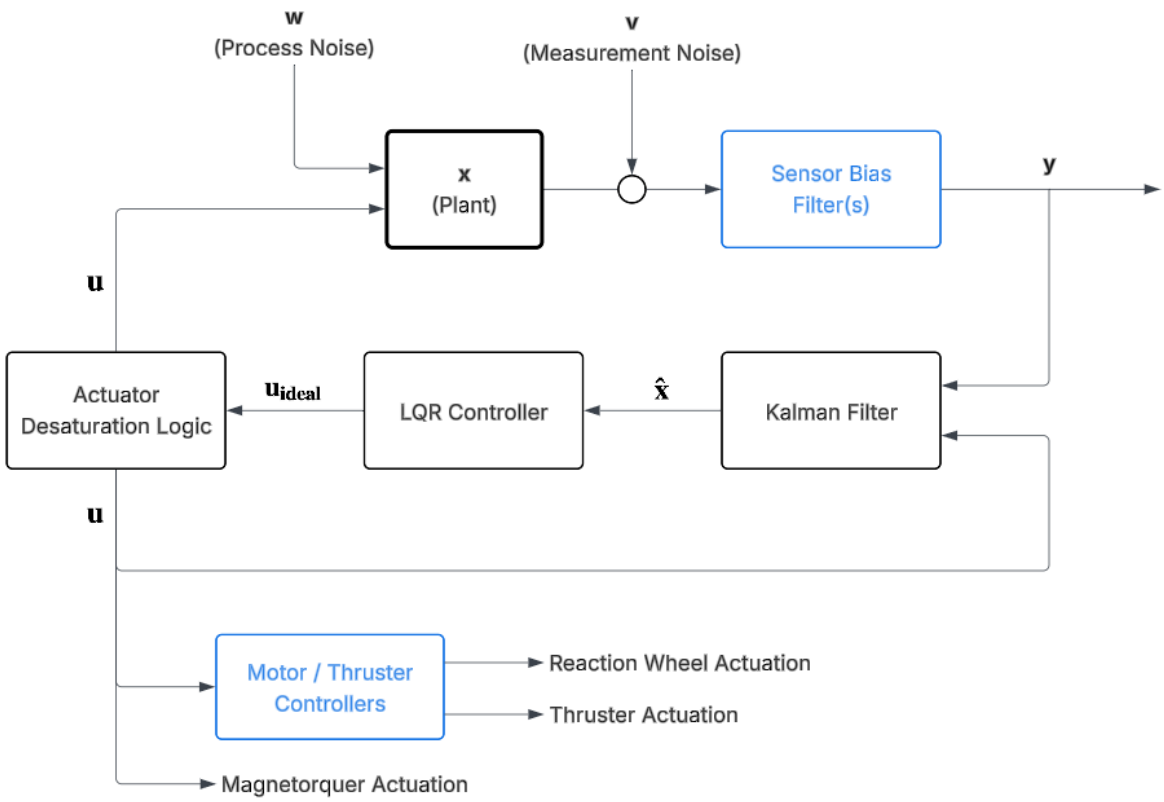

Figure 4.1: OrbitNav Block Diagram. The processes in blue are to be implemented by the user.

5 Performance and Feasibility

5.1 Strategy for High Pointing Accuracy

Orbit Corp's strategy for achieving high pointing accuracy and the dynamic performance necessary for agile maneuvers is twofold:

1. Leverage Kalman filtering to achieve high fidelity state estimation — by optimally combining sensor data with an understanding of the CubeSat's physical dynamics. After all, even a highly optimized baseline (PID) controller will fail if it persistently has a poor understanding of the state of the system that it controls.

2. Use an LQR controller to achieve fast convergence without overly wearing down actuators, while even providing users with a relatively intuitive way to tune the balance between these two objectives via the state weight, $\mathbf{Q}$, and input cost, $\mathbf{R}$, matrices.

5.2 Simulation

Preliminary simulation results, comparing the Orbit Nav module's performance against a baseline PID controller — with the same CubeSat, actuator parameters, and target attitude function — support our claims that our Kalman filter and LQR controller can achieve higher pointing accuracy and better dynamic performance during agile maneuvers.

5.2.1 Simulation Overview

For this simulation, the CubeSat's target attitude [rad] was defined as a function of time [s]:

$$\text{target attitude} = [\sin(t * 10^{-2}), \cos(t * 10^{-2}), \sin(t * 10^{-2}) + \cos(t * 10^{-2})]^T$$

The LQR controller also takes target angular rates [rad/s], which are simply the time derivative of the target attitude function.

The initial state was set away from the initial target attitude, at:

$$x_0 = [0.00, 0.00, 0.00, 0.50, -0.50, 0.50]^T$$

5.2.2 Simulation Result Examples

Note: Attitude and angular rates are now represented in degrees and degrees per second instead of radians and radians per second (respectively) for a more intuitive visualization.

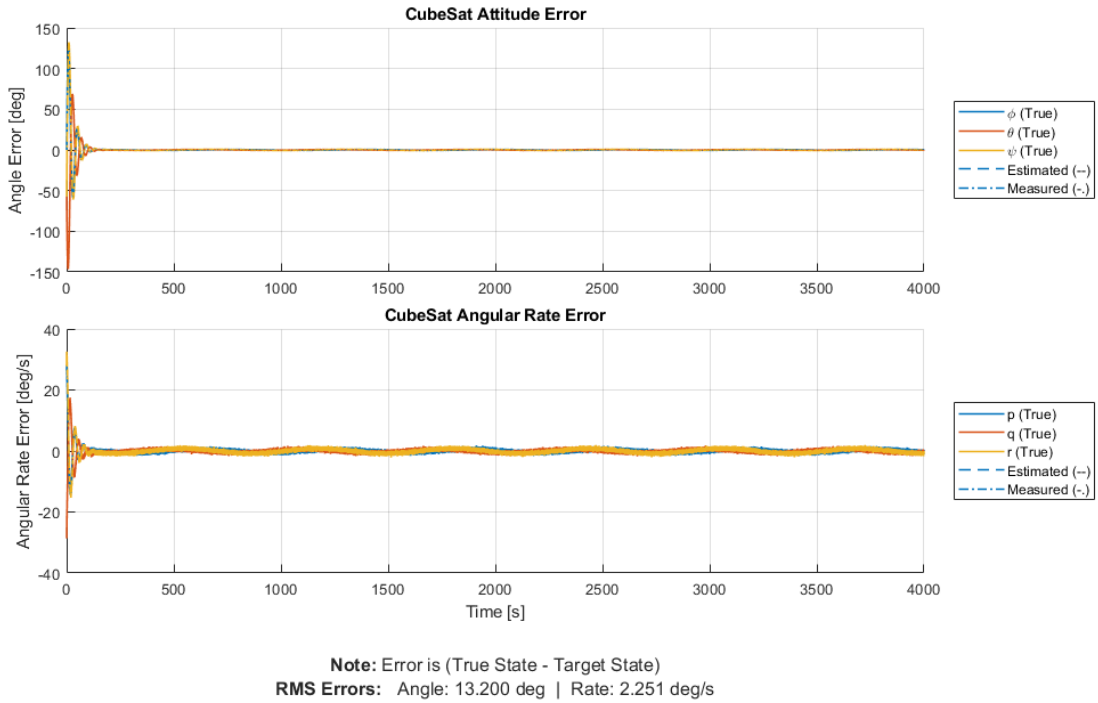

Figure 5.1: Baseline (PID) Attitude Error. The root mean square (RMS) attitude error achieved by the baseline controller was 13.200 degrees.

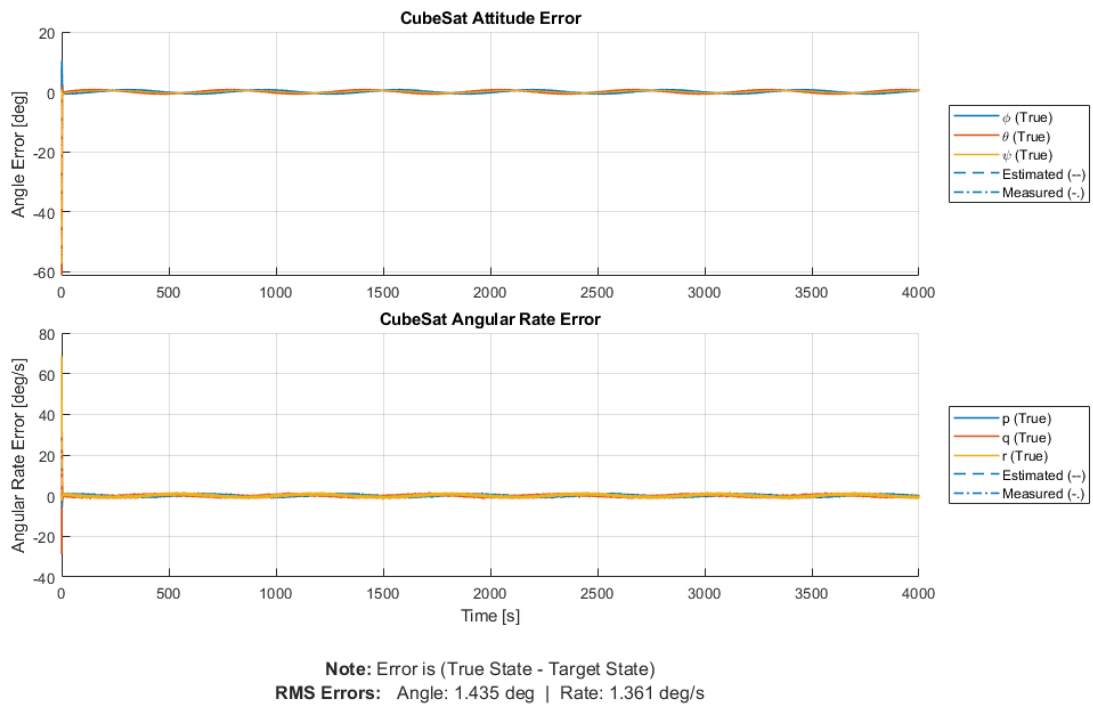

Figure 5.2: OrbitNav (KF + LQR) Attitude Error. The RMS attitude error achieved by the OrbitNav LQR controller was 1.435 degrees.

Based on the results shown in Figures 5.1 and 5.2, the combination of the Kalman filter and LQR controller implemented in OrbitNav achieved an almost 90% reduction in RMS attitude

error from the baseline controller.

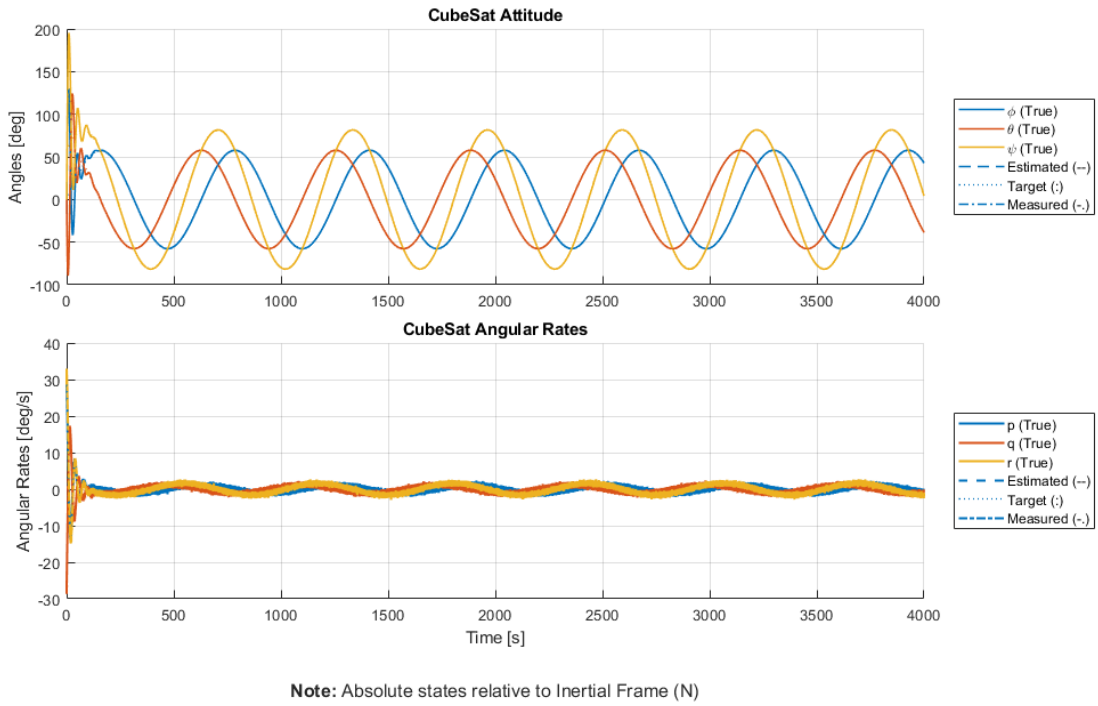

Figure 5.3: Baseline (PID) Attitude and Angular Rates

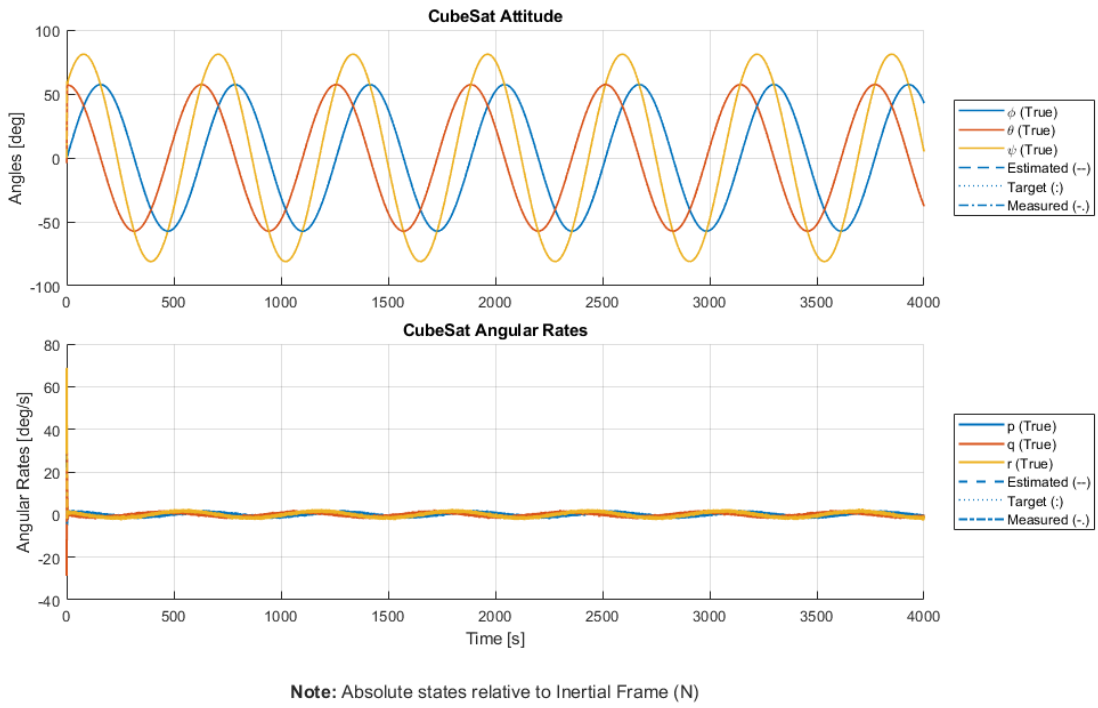

Figure 5.4: OrbitNav (KF + LQR) Attitude and Angular Rates

We see from Figures 5.3 and 5.4 that the OrbitNav controller drives the system to converge with the target attitude faster than the baseline controller.

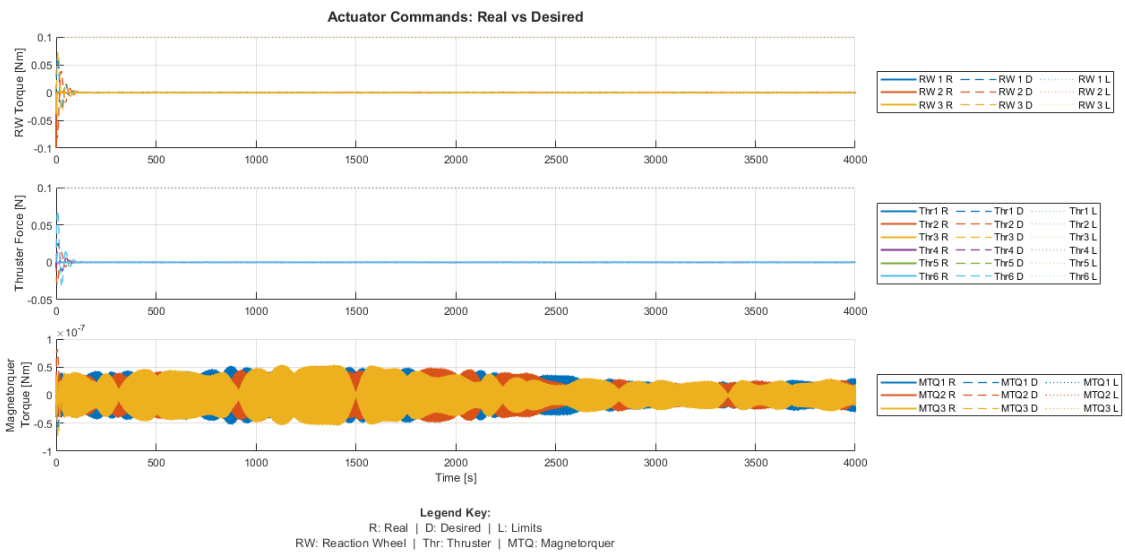

Figure 5.5: Baseline (PID) Actuator Commands

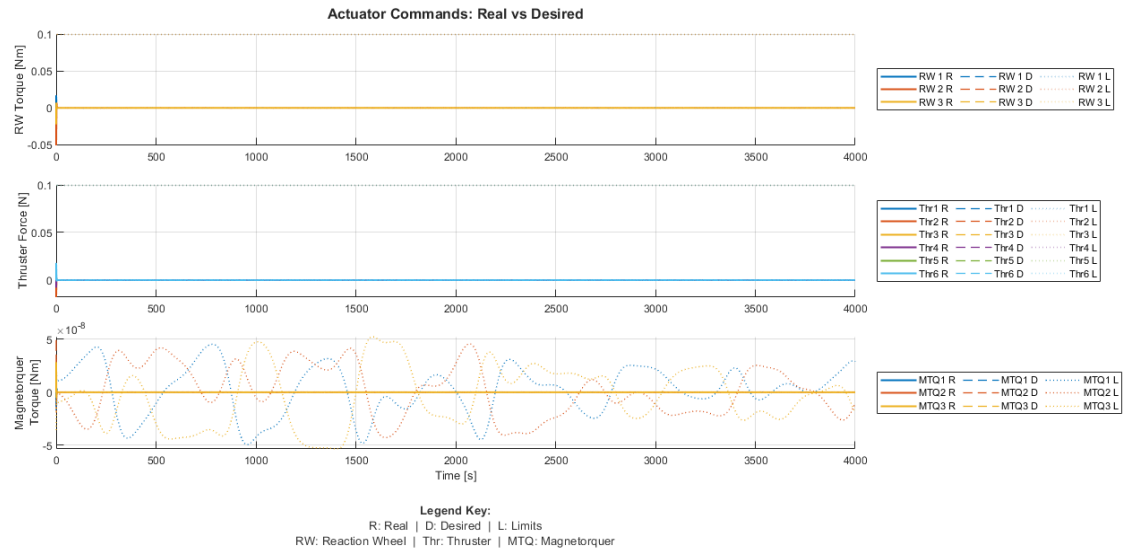

Figure 5.6: OrbitNav (KF + LQR) Actuator Commands

The baseline controller does not have input costs to dictate how it utilizes the actuators. As a result, it "sees" the magnetic torque generated by the magnetorquers as a torque that can be freely generated at will, and thus almost always operates the magnetorquers at their limits while switching the direction of the torque (and hence the current) extremely rapidly — as seen in Figure 5.5. This would put extreme strain on the magnetorquers. By contrast, the OrbitNav's LQR controller is tuned with higher weights on the magnetic torque input, so as to avoid over-relying on the magnetorquers (which only rarely hit their limits in Figure 5.6).

These results suggest that with a good choice of weights and input costs for its LQR controller (which can be determined through simulation), the OrbitNav module can achieve higher pointing accuracy, faster, and more efficiently than a baseline PID controller.